



山东大学

信息科学与工程学院

2024—2025 学年第二学期

实验报告

课程名称: 数据结构与算法导论

实验名称: 实验 6: Dijkstra 算法实现

专 业 班 级 通信工程 三班

学 生 学 号 _____

学 生 姓 名 _____

实 验 时 间 2025 年 5 月 26 日

实验报告

【实验目的】

掌握最短路径算法的基本原理及编程实现；

【实验原理】

Dijkstra 算法用于解决单源最短路径问题（且已被数学证明解决该类问题最优），适用于边权非负的图。基本思想是逐步确定从源点到各顶点的最短路径，通过贪心策略每次选择当前未处理的最近顶点，并更新其邻接顶点的路径长度。

基本步骤如下：

- (1) 若边存在，初始化距离数组为边权,初始化前驱节点为源节点，否则初始化为无穷和-1（不存在前驱）（对于源节点初始化为距离 0 和不存在前驱）。
- (2) 选择与当前节点距离最小的节点 n
- (3) 遍历 n 的所有邻接节点，如果经过 n 的路径更短，更新距离（松弛操作）
- (4) 直到到达终点结束

实现 Dijkstra 算法有两种数据结构，其一为邻接矩阵，大小为顶点²，其空间复杂度较高，矩阵中存放两个顶点的边权。定义如下：

```
struct MGraph {  
    int numVertexes;  
    int arc[MAXVEX][MAXVEX];  
};
```

邻接表则是通过链表表示的图，它的表示方法比邻接矩阵复杂很多，我们需要分别定义出邻接边与顶点节点：（注意，由于是无向图，我们需要构建双向链表）

```
// 边表节点
typedef struct EdgeNode {
    int adjvex;           // 邻接点下标
    int weight;           // 边权重
    struct EdgeNode* next; // 指向下一条边的指针
} EdgeNode;

// 顶点表节点
typedef struct VertexNode {
    int data;             // 顶点数据
    EdgeNode* firstEdge;  // 指向第一条邻接边的指针
} VertexNode;

// 邻接表结构
typedef struct {
    VertexNode adjList[MAXVEX]; // 顶点数组
    int numVertexes;           // 顶点数
} GraphAdjList;
```

我们还需要将顶点与邻接边连接起来，因此我们还需要完成加边函数：

```
void AddEdge(GraphAdjList* G, int src, int dest, int weight) {
    // 创建边节点并添加到源顶点的邻接表头部
    EdgeNode* e = (EdgeNode*)malloc(sizeof(EdgeNode));
    e->adjvex = dest;
    e->weight = weight;
    e->next = G->adjList[src].firstEdge;
    G->adjList[src].firstEdge = e;
}
```

【实验内容与要求】

内容：

- 1) 建立一张图，选择一种存储结构(邻接矩阵或邻接表)初始化该图；
- 2) 用Dijkstra算法实现点与点之间的最短路径

要求：

- 1) 实现图的两种表示方法；
- 2) 实现Dijkstra算法。

【实验源程序】

源程序：

```
/* 邻接表结构定义 */
#include <stdio.h>
#include <stdlib.h>

// 定义最大顶点数为 9, INFINITY 表示无穷大（不可达）
#define MAXVEX 9
#define INFINITY 65535

// 自定义类型: Patharc 用于存储最短路径上的前驱节点, ShortPathTable 存储到各顶点的最短路径长度
typedef int Patharc[MAXVEX];
typedef int ShortPathTable[MAXVEX];

/* 邻接矩阵结构定义 */
struct MGraph {
    int numVertexes;           // 顶点数
    int arc[MAXVEX][MAXVEX];  // 邻接矩阵, 存储边权重
};

/* 邻接表结构定义 */
// 边表节点
typedef struct EdgeNode {
    int adjvex;               // 邻接点下标
    int weight;               // 边权重
    struct EdgeNode* next;    // 指向下一条边的指针
} EdgeNode;

// 顶点表节点
typedef struct VertexNode {
    int data;                 // 顶点数据
    EdgeNode* firstEdge;      // 指向第一条邻接边的指针
} VertexNode;

// 邻接表结构
typedef struct {
    VertexNode adjList[MAXVEX]; // 顶点数组
    int numVertexes;           // 顶点数
```

```

} GraphAdjList;

/**
 * 邻接矩阵的 Dijkstra 算法实现
 * 参数 G    邻接矩阵图指针
 * 参数 v0    源点
 * 参数 vv    目标顶点
 * 参数 P     前驱节点数组
 * 参数 D     最短路径长度数组
 */
void Dijkstra_Matrix(MGraph* G, int v0, int vv, Patharc P, ShortPathTable D) {
    int found[MAXVEX] = {0};    // 标记顶点是否找到最短路径

    // 初始化 D 和 P 数组
    for (int v = 0; v < G->numVertexes; ++v) {
        D[v] = G->arc[v0][v];    // 初始化为源点直接到各顶点的距离
        P[v] = (D[v] < INFINITY) ? v0 : -1; // 存在边则前驱为 v0, 否则为-1
    }

    D[v0] = 0;    // 源点到自身的距离为 0
    found[v0] = 1;    // 标记源点已处理
    P[v0] = -1;    // 源点无前驱

    // 循环处理所有顶点, 每次找到一个顶点的最短路径
    for (int i = 1; i < G->numVertexes; ++i) {
        int min = INFINITY, k = -1;
        // 寻找当前未处理顶点中距离最小的顶点 k
        for (int w = 0; w < G->numVertexes; ++w) {
            if (!found[w] && D[w] < min) {
                min = D[w];
                k = w;
            }
        }

        if (k == -1) break; // 所有顶点已处理或不可达, 退出
        found[k] = 1;    // 标记顶点 k 已处理
        if (k == vv) break; // 提前退出: 已找到目标顶点的最短路径

        // 更新 k 的邻接顶点的最短路径
        for (int w = 0; w < G->numVertexes; ++w) {
            // 若经过 k 到 w 的距离更短, 则更新 D[w] 和 P[w]
            if (!found[w] && (min + G->arc[k][w] < D[w])) {
                D[w] = min + G->arc[k][w];
            }
        }
    }
}
    
```

```

        P[w] = k;
    }
}

/* 输出最短路径信息 */
if (D[vv] == INFINITY) {
    printf("Matrix: No path found!\n");
    return;
}

// 逆向回溯路径并存储到 path 数组
int path[MAXVEX], cnt = 0;
for (int cur = vv; cur != -1; cur = P[cur])
    path[cnt++] = cur;

// 正向打印路径
printf("Matrix Path: ");
for (int i = cnt - 1; i >= 0; --i)
    printf(i ? "%d->" : "%d", path[i]);
printf("\nLength: %d\n\n", D[vv]);
}

/**
 * 向邻接表添加边（双向）
 * 参数 G      邻接表指针
 * 参数 src    源顶点
 * 参数 dest   目标顶点
 * 参数 weight 边权重
 */
void AddEdge(GraphAdjList* G, int src, int dest, int weight) {
    // 创建边节点并添加到源顶点的邻接表头部
    EdgeNode* e = (EdgeNode*)malloc(sizeof(EdgeNode));
    e->adjvex = dest;
    e->weight = weight;
    e->next = G->adjList[src].firstEdge;
    G->adjList[src].firstEdge = e;
}

/**
 * 邻接表的 Dijkstra 算法实现
 * 参数 G      邻接表图指针
 * 参数 v0     源点

```

```

* 参数 vv    目标顶点
* 参数 P     前驱节点数组
* 参数 D     最短路径长度数组
*/
void Dijkstra_AdjList(GraphAdjList* G, int v0, int vv, int* P, int* D) {
    int found[MAXVEX] = {0}; // 标记顶点是否已找到最短路径

    // 初始化 D 和 P 数组
    for (int i = 0; i < G->numVertexes; ++i) {
        D[i] = INFINITY;
        P[i] = -1;
    }
    D[v0] = 0; // 源点到自身距离为 0

    // 处理所有顶点
    for (int i = 0; i < G->numVertexes; ++i) {
        int min = INFINITY, k = -1;
        // 寻找当前未处理顶点中距离最小的顶点 k
        for (int j = 0; j < G->numVertexes; ++j) {
            if (!found[j] && D[j] < min) {
                min = D[j];
                k = j;
            }
        }

        if (k == -1) break; // 所有顶点处理完毕或不可达
        found[k] = 1;      // 标记 k 已处理
        if (k == vv) break; // 提前退出: 已找到目标顶点

        // 遍历 k 的所有邻接边, 更新邻接顶点的最短路径
        EdgeNode* e = G->adjList[k].firstEdge;
        while (e) {
            int w = e->adjvex;
            if (!found[w] && D[k] + e->weight < D[w]) {
                D[w] = D[k] + e->weight;
                P[w] = k;
            }
            e = e->next;
        }
    }

    /* 输出最短路径信息 */
    if (D[vv] == INFINITY) {

```

```

        printf("AdjList: No path found!\n");
        return;
    }

    // 逆向回溯路径并存储到 path 数组
    int path[MAXVEX], cnt = 0;
    for (int cur = vv; cur != -1; cur = P[cur])
        path[cnt++] = cur;

    // 正向打印路径
    printf("AdjList Path: ");
    for (int i = cnt - 1; i >= 0; --i)
        printf(i ? "%d->" : "%d", path[i]);
    printf("\nLength: %d\n", D[vv]);
}

/**
 * 初始化邻接矩阵图（无向图）
 * 参数 G 邻接矩阵图指针
 */
void InitMGraph(MGraph* G) {
    G->numVertexes = 9; // 设置顶点数为 9

    // 初始化所有边为 INFINITY，对角线为 0
    for (int i = 0; i < MAXVEX; ++i) {
        for (int j = 0; j < MAXVEX; ++j) {
            G->arc[i][j] = INFINITY;
            if (i == j) G->arc[i][j] = 0;
        }
    }

    // 设置各边的权重（无向图，对称赋值）
    G->arc[0][1] = G->arc[1][0] = 1;
    G->arc[0][2] = G->arc[2][0] = 5;
    G->arc[1][2] = G->arc[2][1] = 3;
    G->arc[1][3] = G->arc[3][1] = 7;
    G->arc[1][4] = G->arc[4][1] = 5;
    G->arc[2][4] = G->arc[4][2] = 1;
    G->arc[2][5] = G->arc[5][2] = 7;
    G->arc[3][4] = G->arc[4][3] = 2;
    G->arc[3][6] = G->arc[6][3] = 3;
    G->arc[4][5] = G->arc[5][4] = 3;
    G->arc[4][6] = G->arc[6][4] = 6;
}

```

```

        G->arc[4][7] = G->arc[7][4] = 9;
        G->arc[5][7] = G->arc[7][5] = 5;
        G->arc[6][7] = G->arc[7][6] = 2;
        G->arc[6][8] = G->arc[8][6] = 7;
        G->arc[7][8] = G->arc[8][7] = 4;
    }

    /**
     * 初始化邻接表图（无向图）
     * 参数 G 邻接表指针
     */
    void InitGraphAdjList(GraphAdjList* G) {
        G->numVertexes = MAXVEX; // 顶点数设为 MAXVEX

        // 初始化每个顶点的邻接表
        for (int i = 0; i < MAXVEX; ++i) {
            G->adjList[i].data = i; // 顶点数据设为索引值
            G->adjList[i].firstEdge = NULL; // 初始无邻接边
        }

        // 添加所有边（无向图，双向添加）
        AddEdge(G, 0, 1, 1); AddEdge(G, 1, 0, 1);
        AddEdge(G, 0, 2, 5); AddEdge(G, 2, 0, 5);
        AddEdge(G, 1, 2, 3); AddEdge(G, 2, 1, 3);
        AddEdge(G, 1, 3, 7); AddEdge(G, 3, 1, 7);
        AddEdge(G, 1, 4, 5); AddEdge(G, 4, 1, 5);
        AddEdge(G, 2, 4, 1); AddEdge(G, 4, 2, 1);
        AddEdge(G, 2, 5, 7); AddEdge(G, 5, 2, 7);
        AddEdge(G, 3, 4, 2); AddEdge(G, 4, 3, 2);
        AddEdge(G, 3, 6, 3); AddEdge(G, 6, 3, 3);
        AddEdge(G, 4, 5, 3); AddEdge(G, 5, 4, 3);
        AddEdge(G, 4, 6, 6); AddEdge(G, 6, 4, 6);
        AddEdge(G, 4, 7, 9); AddEdge(G, 7, 4, 9);
        AddEdge(G, 5, 7, 5); AddEdge(G, 7, 5, 5);
        AddEdge(G, 6, 7, 2); AddEdge(G, 7, 6, 2);
        AddEdge(G, 6, 8, 7); AddEdge(G, 8, 6, 7);
        AddEdge(G, 7, 8, 4); AddEdge(G, 8, 7, 4);
    }

    int main() {
        // 初始化邻接矩阵图并执行 Dijkstra 算法
        MGraph g_matrix;
        InitMGraph(&g_matrix);
    }

```

```

Patharc P_matrix;
ShortPathTable D_matrix;
printf("邻接矩阵结果:\n");
Dijkstra_Matrix(&g_matrix, 0, 8, P_matrix, D_matrix); // 计算 0 到 8 的最短路径

// 初始化邻接表图并执行 Dijkstra 算法
GraphAdjList g_adjlist;
InitGraphAdjList(&g_adjlist);
int P_adjlist[MAXVEX], D_adjlist[MAXVEX];
printf("邻接表结果:\n");
Dijkstra_AdjList(&g_adjlist, 0, 8, P_adjlist, D_adjlist);

return 0;
}

```

【实验结果】

我们构建的图结构如下：

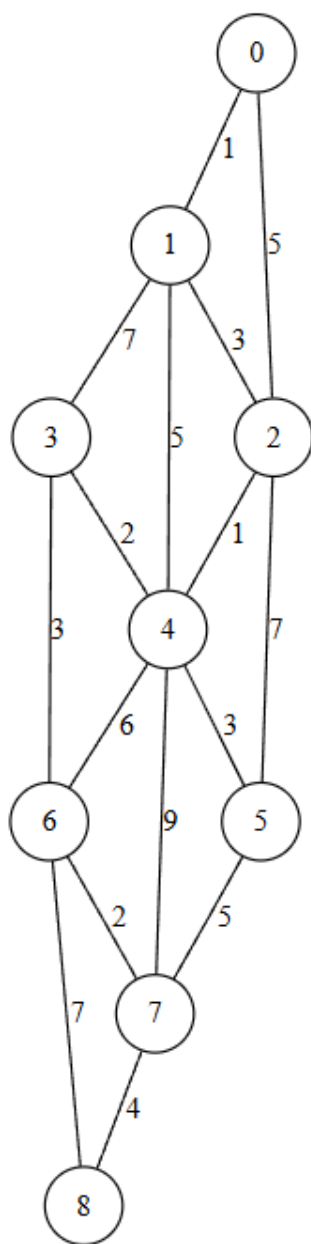
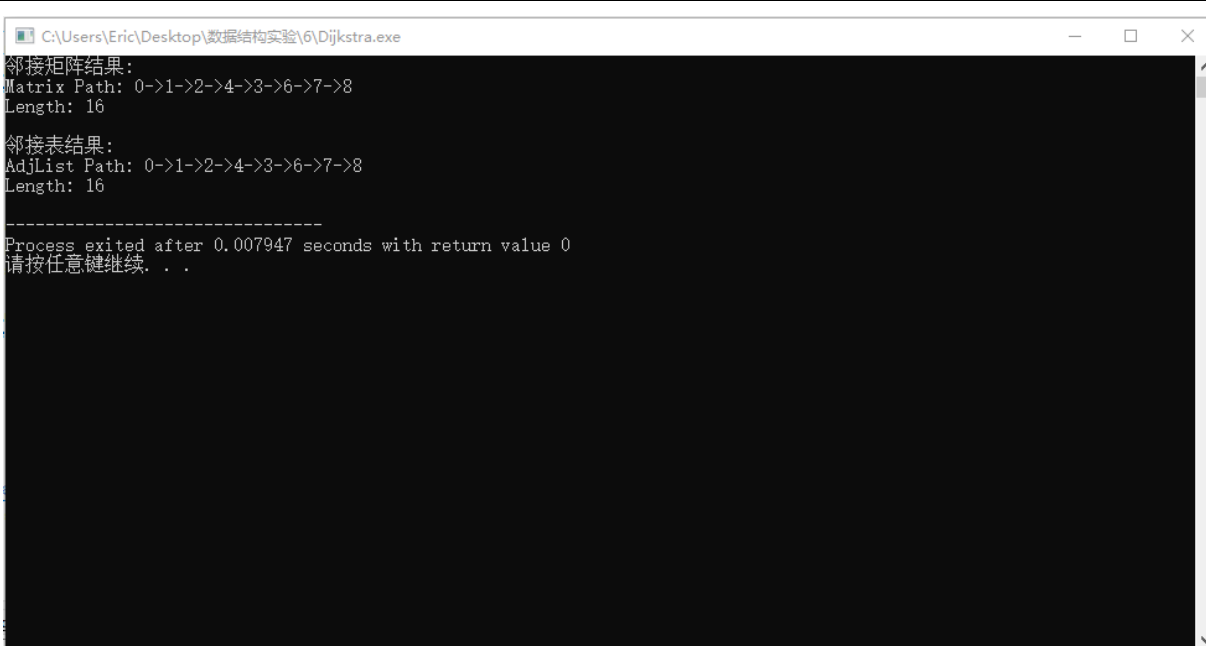


图 1 构造出来的图

我们通过两种数据结构得到的结果为：



```

C:\Users\Eric\Desktop\数据结构实验\6\Dijkstra.exe
邻接矩阵结果:
Matrix Path: 0->1->2->4->3->6->7->8
Length: 16

邻接表结果:
AdjList Path: 0->1->2->4->3->6->7->8
Length: 16

-----
Process exited after 0.007947 seconds with return value 0
请按任意键继续. . .
    
```

图 2 运行结果

通过我们手动验算，该路径权已到达最小，说明我们的实验成功完成了 Dijkstra 算法。

【实验心得】

本次实验，我们成功通过邻接矩阵与邻接表实现了 Dijkstra 算法。也深刻地理解了这两种数据结构，在不同情况下的应用场景。对于邻接矩阵，如果将其运用在顶点数众多而边数非常少的场景下，这会大大地浪费空间存储。而邻接表如果用在顶点数多而边数也多的情况下，邻接表的操作与描述也会相当复杂，因此，在图非常稠密的情况下应使用邻接矩阵，非常稀疏的情况下应该使用邻接表。

同样的，我在本次实验中也遇到不少的问题，其中在进行初始化操作时，我忘记给源节点的前驱与标记已处理，导致输出不了路径。

正确的处理方法如下：

```

D[v0] = 0;           // 源点到自身的距离为 0
found[v0] = 1;       // 标记源点已处理
P[v0] = -1;          // 源点无前驱
    
```

还有一些可以优化的地方，在实现寻找最小的元素的算法中，我选择的算法是直接遍历寻找最小值，时间复杂度为 $O(n^2)$ ，但是实际上，我们可以使用最小堆来完成这个部分。这样时间复杂度

可以降至 $O(n \log n)$